

Distributed Job Scheduler

Technical Design Document

Reliable background job execution across multiple workers

Candidate	Hari R
Registration no.	127156127
Submitted to	Codity.AI — Intern Technical Assignment
Date	21 August 2026
Repository	https://github.com/Hari-R1506/distributed-job-scheduler
Stack	TypeScript · NestJS · PostgreSQL 16 · React · Docker
Scale	16 tables · 56 API operations · 7 containers · 85 unit tests

Contents

Deliverables	4
Evaluating it in five minutes	4
1. Executive summary	5
1.1 The problem, precisely	5
1.2 The governing design decision	5
1.3 What was built	5
2. System architecture	6
2.1 Why a modular monolith, and one genuine split	6
2.2 Leader election, and why it is structural	6
2.3 Job discovery: push for latency, polling for correctness	6
3. Database design	8
3.1 The split that matters most	8
3.2 The claim index	9
3.3 Cascade rules are chosen per relationship	9
3.4 Three deliberate denormalisations	9
3.5 Invalid states are unrepresentable	10
4. Atomic job claiming	11
4.1 The problem	11
4.2 Approaches considered	11
4.3 How SKIP LOCKED works	11
4.4 The claim query	11
4.5 Priority: the trap in the brief	12
5. Job lifecycle and crash recovery	14
5.1 Where "Failed" went	14
5.2 The conditional write	14
5.3 Crash recovery	14
5.4 Heartbeats and the timing invariant	15
5.5 Graceful shutdown	16
6. Retries and backoff	17
6.1 The three strategies	17
6.2 Jitter is not decoration	17
6.3 The worker does not sleep	18
6.4 Not every failure deserves five attempts	18
7. Dead letter queue	18
7.1 What happens, transactionally	18
7.2 Triage by error signature	18
7.3 Replay creates a new job	18
8. Idempotency	20
8.1 Duplicate submission — fully solvable	20
8.2 Duplicate execution — not fully solvable	20
9. Design decisions and trade-offs	21
9.11 Deliberately not built	22
10. API design	23

10.1 Conventions.....	23
10.2 No POST /workers.....	23
11. Testing strategy.....	24
11.1 Unit tests — 85, no database, under one second.....	24
11.2 Concurrency tests — the gate	24
11.3 Defects found by this discipline	24
12. Running the system.....	26
12.1 One command	26
12.2 The demonstration	26
12.3 Verified state	26
12.4 Where the detail lives.....	27

Deliverables

Every item required by the assignment, and where it is. All links point into the public repository.

#	Required	Where it lives
1	Source code with setup instructions	The repository · README.md · docs/SETUP.md
2	Architecture diagram	Section 2 of this document · docs/ARCHITECTURE.md (6 diagrams)
3	ER diagram	Section 3 of this document · docs/DATABASE.md (16 tables)
4	API documentation	Section 10 · docs/API.md · live Swagger at /docs · docs/api/openapi.json
5	Design decisions document	Section 9 of this document · docs/DESIGN-DECISIONS.md
6	Automated tests	Section 11 · tests/ — unit, integration and concurrency suites

Repository:

<https://github.com/Hari-R1506/distributed-job-scheduler>

Evaluating it in five minutes

```
git clone https://github.com/Hari-R1506/distributed-job-scheduler.git
cd distributed-job-scheduler
cp .env.example .env
npm install && npm run db:generate
docker compose up -d --build
```

Register an account at <http://localhost:5173>, then run `npm run seed`. Seven containers start: PostgreSQL, the API, a leader-elected scheduler, three independent workers, and the dashboard.

The demonstration

Open the Workers page in the dashboard, then:

```
docker kill djs-worker-2
```

SIGKILL — instant termination, no drain, no lease release. Equivalent to a server losing power mid-job.

- **At roughly 30 seconds** the scheduler marks the worker DEAD after six missed heartbeats.
- **At roughly 60 seconds** the reaper finds its expired leases, closes those attempts as ABANDONED, and requeues the jobs.
- **Immediately after** the surviving workers claim and complete them.

Nothing is lost, and nobody intervenes. Contrast with `docker stop`, which sends SIGTERM: the worker drains its in-flight jobs and exits cleanly in about three seconds, with no job retried.

1. Executive summary

This document describes a production-inspired distributed job scheduling platform: a system that accepts background work through a REST API, executes it reliably across multiple independent worker processes, and recovers automatically when any of those processes dies.

1.1 The problem, precisely

What separates this from an ordinary CRUD application is a single sentence: multiple workers read from the same list at the same time, and any of them can die at any moment.

Everything difficult in this design follows from that sentence:

- Two workers must never run the same job → **atomic claiming** (§4)
- A dead worker must not strand its work → **leases, heartbeats and a reaper** (§5)
- A failing job must return later, not immediately → **backoff scheduling, never sleeping** (§6)
- A job that can never succeed must stop consuming capacity → **a dead letter queue** (§7)
- A retried job must not double-charge anyone → **idempotency** (§8)

1.2 The governing design decision

The database is the queue. Workers hold short transactions to claim work, and hold no transaction while doing work.

Every reliability property described in this document is a consequence of that rule. PostgreSQL provides the primitives that make it viable: FOR UPDATE SKIP LOCKED as a genuine work-queue operation, advisory locks for leader election, LISTEN/NOTIFY for push wake-ups, and partial indexes that keep the claim path fast as the table grows to millions of rows.

Deliberate omission — No message broker was introduced. A broker would split the system in two — creating a job would mean writing to PostgreSQL and pushing to the broker, with no transaction spanning both. That dual-write problem buys throughput this system will never need. The reasoning is set out in full in §9.1.

1.3 What was built

Component	Responsibility
API service(NestJS)	Authentication, tenancy, validation, job and queue management, read models. 56 operations. Never executes a job.
Worker(N processes)	Claims jobs atomically, executes them concurrently, heartbeats, drains gracefully on SIGTERM.
Scheduler(leader-elected)	Promotes due jobs, fires cron schedules, reaps expired leases, rolls up metrics. Exactly one instance acts at a time.
Dashboard(React)	Overview, queues, job explorer, worker health, dead-letter triage.
PostgreSQL 16	The system of record and the queue itself.

2. System architecture

Three deployables, one codebase, one database. The API is a modular monolith; the workers are a separately scaled fleet; the scheduler is the same binary running in a different role.

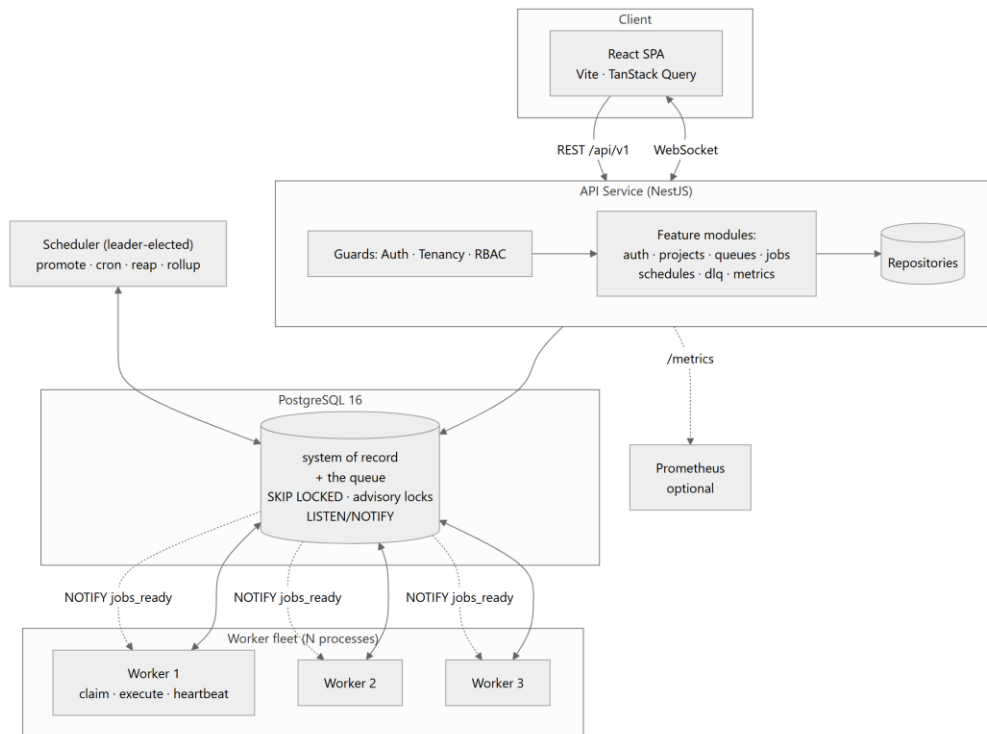


Figure 1 — System architecture

2.1 Why a modular monolith, and one genuine split

The API modules share one database and one transaction boundary. Splitting them into services would require distributed transactions to create a job and its audit trail atomically — strictly worse, for no benefit at this scale.

The worker, however, is genuinely separate, and that is the one split the problem justifies. It has a different scaling axis (add workers, not API capacity), a different failure domain (a handler that exhausts memory must not take down the dashboard), and a different lifecycle (a 30-second graceful drain versus an instant restart).

2.2 Leader election, and why it is structural

Three of the scheduler's loops — promotion, cron materialisation, and lease reaping — are globally singular. Running two copies fires every cron schedule twice.

The obvious approach is a dedicated deployable that operators must remember to run exactly one of. That works until somebody scales it to two. Instead, every process attempts `pg_try_advisory_lock` at startup; whichever wins runs the scheduler loops and the rest idle as hot standbys. Correctness stops depending on deployment discipline.

Why this matters — *Failover costs nothing. If the leader dies, its session ends, PostgreSQL releases the advisory lock automatically, and the next process to retry becomes leader — no lease to expire, no heartbeat to miss, no split brain. This also satisfies the assignment's "distributed locking" bonus as load-bearing architecture rather than a bolt-on.*

2.3 Job discovery: push for latency, polling for correctness

Pure polling wastes queries and adds latency equal to half the poll interval. Pure push loses jobs when a notification fires while a worker is reconnecting. The system uses both: LISTEN/NOTIFY wakes workers immediately, and a jittered poll timer runs underneath it.

NOTIFY is the latency optimisation; the poll timer is the correctness guarantee. **If every notification were lost, the system would still be correct — only slower.** That is the right way round, and it is why no delivery guarantee is needed from a mechanism that offers none.

PostgreSQL 16. **16 tables** · **9 enum types** · **36 foreign keys** · **14 CHECK constraints** · **8 hand-written partial indexes**.



`jobs` holds the logical unit of work. `job_executions` holds one row per *attempt*. **Attempts fail; jobs die.**

Distributed Job Scheduler · 8

3.2 The claim index

This is the single highest-value object in the schema.

```
CREATE INDEX idx_jobs_claim
ON jobs (queue_id, priority DESC, run_at ASC, id ASC)
WHERE status = 'QUEUED';
```

Four properties, all load-bearing:

- **It is partial.** After a week of operation roughly 99% of rows are terminal. A full index would carry all of them; this one carries only the ready working set — so claim cost depends on *queue depth*, not *table size*.
- **queue_id is first** because it is the equality predicate, selecting the sub-tree the scan walks.
- **The remaining columns are the ORDER BY, in order.** PostgreSQL walks the index and stops at LIMIT. Without that alignment the planner must read every eligible row and sort — turning an $O(n)$ claim into $O(N \log N)$ under load.
- **id is a final tiebreaker**, making the ordering total and deterministic so concurrency tests are reproducible.

Measured, not asserted

Seeded with 50,000 jobs across 4 queues, 5,000 still QUEUED — the realistic shape where most rows are terminal:

```
Limit (actual time=0.030..0.066 rows=10 loops=1)
-> LockRows (actual time=0.030..0.065 rows=10)
    -> Index Scan using idx_jobs_claim on jobs
        Index Cond: (queue_id = ... AND run_at <= now())
        Filter: (status = 'QUEUED'::job_status)
        Buffers: shared hit=12
Execution Time: 0.098 ms
```

There is **no Sort node**. Twelve buffer hits and 0.098 ms to select ten jobs from fifty thousand. The claim path costs the same whether the table holds 50,000 rows or 50 million.

3.3 Cascade rules are chosen per relationship

Action	Count	Applied to
CASCADE	21	Ownership chains only: org → project → queue → job → execution → log. Deleting an organization removes what it owns and nothing else.
SET NULL	13	Attribution and provenance: created_by, paused_by, job_executions.worker_id. Deleting a user must never block, and execution history must survive a worker row being purged.
RESTRICT	1	queues.retry_policy_id. Deleting a policy that queues depend on fails loudly rather than silently leaving queues undefined. The API surfaces this as 409, not a 500.

3.4 Three deliberate denormalisations

Denormalisation	Justification
jobs.project_id	Derivable through queue_id, but every tenant-scoped list query filters on it. Carrying it lets the explorer index lead with project_id. The value is immutable, so there is no update anomaly.
Retry policy copied onto each job	Correctness, not performance. A job's retry contract is fixed at submission. Reading the queue's policy live would mean editing it silently rewrites the behaviour of thousands of in-flight jobs, including ones already mid-backoff.

Denormalisation	Justification
dlq.payload_snapshot	Retention may later purge the original payload. A dead-letter entry that cannot be replayed is worthless.

3.5 Invalid states are unrepresentable

Each of the following was attempted against the live database and rejected:

Attempt	Rejected by
Set a job to RUNNING with no lease	chk_jobs_lease_present
Set priority = 999	chk_jobs_priority
Register Alice@Example.com and alice@example.com	unique violation on citext

Why — *The first is the one that matters. A job stranded in RUNNING with no lease_expires_at would be invisible to the reaper and never recovered — a silent, permanent leak. The constraint converts that class of bug from a production mystery into a failing test.*

4. Atomic job claiming

This is the core of the assignment. Everything else is scaffolding around this section.

4.1 The problem

Worker A and Worker B poll the same queue in the same millisecond. Job #101 is QUEUED and ready.

```
t0  A: SELECT ... WHERE status='QUEUED' -> sees #101
t0  B: SELECT ... WHERE status='QUEUED' -> sees #101 <- both
t1  A: UPDATE jobs SET status='CLAIMED', worker_id=A WHERE id=101
t1  B: UPDATE jobs SET status='CLAIMED', worker_id=B WHERE id=101
t2  Both execute the job. The customer is charged twice.
```

Wrapping this in a transaction under PostgreSQL's default **READ COMMITTED** isolation does *not* fix it: B's SELECT sees a snapshot taken before A's uncommitted UPDATE, so B still sees #101 as available.

4.2 Approaches considered

Approach	Correct?	Verdict
SELECT then UPDATE(READ COMMITTED)	No	The bug above.
SERIALIZABLE isolation	Yes	Every concurrent claim conflicts and aborts with 40001. All N workers serialise and N-1 retry. Throughput collapses exactly when workers are added.
SELECT ... FOR UPDATE(no SKIP LOCKED)	Yes	B blocks on A's row lock, then wakes to find the row no longer matches. Workers queue behind each other — a convoy.
Advisory lock per job id	Yes	Requires a lock per candidate row; lock-table pressure and awkward release semantics.
FOR UPDATE SKIP LOCKED	Yes	CHOSEN. No blocking, no aborts, throughput scales linearly with worker count.

4.3 How SKIP LOCKED works

Three mechanisms combine:

- **Row-level write locks.** SELECT ... FOR UPDATE locks each returned row until the transaction ends.
- **SKIP LOCKED changes the conflict behaviour.** Normally a locked row makes you wait. With SKIP LOCKED the executor steps over it and continues scanning. A locks #101; B — scanning the same index at the same instant — simply does not see it and takes #102.
- **The UPDATE rides on the same locks** in the same transaction, so nothing can touch those rows in between.

```
t0  A: BEGIN; SELECT ... FOR UPDATE SKIP LOCKED LIMIT 5 -> [101..105]
t0  B: BEGIN; SELECT ... FOR UPDATE SKIP LOCKED LIMIT 5 -> [106..110]
t1  A: UPDATE those 5 -> CLAIMED by A; COMMIT
t1  B: UPDATE those 5 -> CLAIMED by B; COMMIT
```

Zero contention, zero duplicates, and throughput that grows with worker count.

4.4 The claim query

Executed inside one short transaction — typically 1 to 3 milliseconds — per queue, per poll.

```

-- Step 1: serialise the claim DECISION for this queue.
SELECT pg_advisory_xact_lock(hashtextextended('queue_claim:' || $1, 0));

-- Step 2-4: capacity, selection, claim. One statement.
WITH capacity AS (
  SELECT CASE
    WHEN q.is_paused THEN 0
    WHEN q.max_concurrency IS NULL THEN $3
    ELSE GREATEST(0, LEAST($3, q.max_concurrency - (
      SELECT count(*) FROM jobs r
      WHERE r.queue_id = q.id
      AND r.status IN ('CLAIMED', 'RUNNING'))))
  END AS n
  FROM queues q WHERE q.id = $1
),
eligible AS (
  SELECT j.id FROM jobs j
  WHERE j.queue_id = $1
  AND j.status = 'QUEUED'
  AND j.run_at <= now() -- ELIGIBILITY, not ranking
  ORDER BY j.priority DESC, j.run_at ASC, j.id ASC
  FOR UPDATE SKIP LOCKED
  LIMIT (SELECT n FROM capacity)
)
UPDATE jobs j
  SET status = 'CLAIMED',
      worker_id = $2,
      claimed_at = now(),
      lease_expires_at = now() + $4,
      attempt_count = j.attempt_count + 1
  FROM eligible e WHERE j.id = e.id
RETURNING j.*;

```

Why the advisory lock is necessary

SKIP LOCKED prevents two workers taking the same row. It cannot enforce the per-queue concurrency limit, because that limit is a constraint over an aggregate — and aggregates are not lockable.

Under READ COMMITTED, two workers both read `count(running) = 0` from snapshots taken before either commits, both compute "3 slots free", and both claim three. Six jobs run on a queue limited to three. There is no row-level conflict to detect, because they locked different rows.

Scoped narrowly — The lock is transaction-scoped, so it cannot be leaked; keyed per queue, so the email queue never blocks the reports queue; and held for microseconds — the duration of the claim, never the duration of execution. Cost: claims on one queue serialise, capping it at roughly 300–1000 claim transactions per second. Each claim takes a batch, so real throughput is many multiples of that.

Isolation level

READ COMMITTED — the default — is correct here. The locking is explicit, so the database does not need to infer conflicts, and **SERIALIZABLE's serialisation failures are avoided entirely**. "I used the default isolation level because my locking is explicit" is a stronger position than "I used SERIALIZABLE to be safe".

4.5 Priority: the trap in the brief

The assignment asks that *a HIGH priority job scheduled for tomorrow must not execute before a LOW priority job that is ready now*. The answer is one distinction:

Eligibility is a WHERE clause. Priority is an ORDER BY clause. They must never be mixed.

Because `run_at <= now()` filters before the sort, tomorrow's CRITICAL job is not in the candidate set at all. It cannot outrank anything, because it is not competing yet. Sorting by `(priority, run_at)` across all jobs — the common mistake — would let it win.

5. Job lifecycle and crash recovery

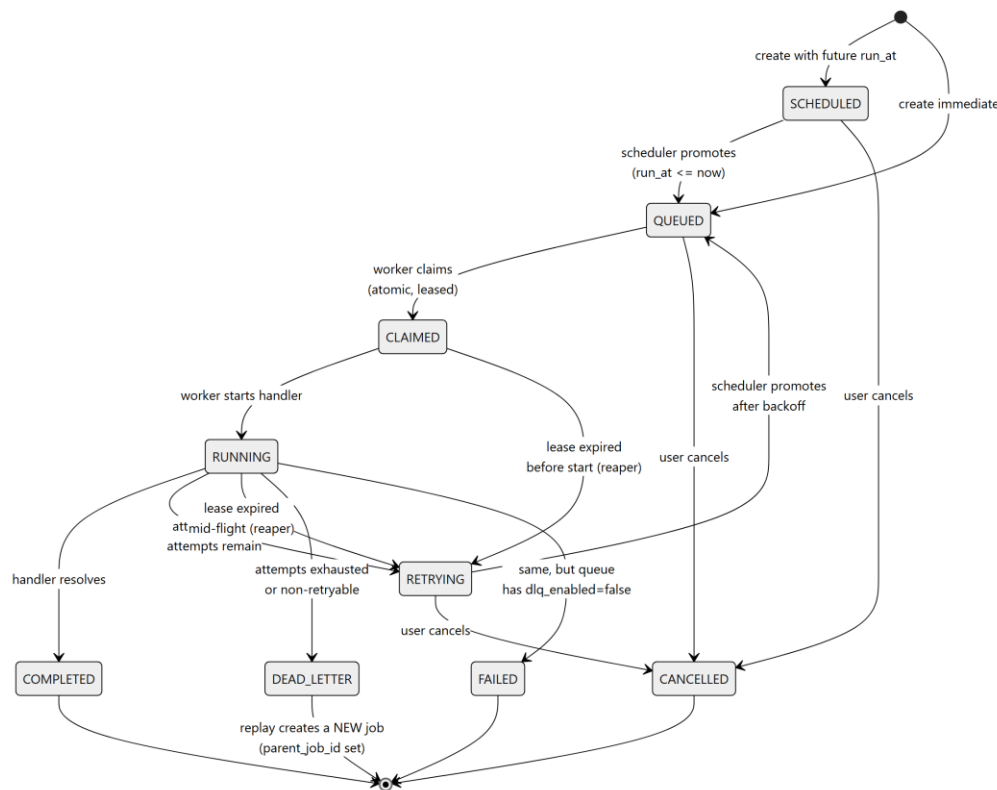


Figure 3 — Job lifecycle state machine

5.1 Where "Failed" went

The brief lists Failed among the lifecycle states. In this model, attempts fail and jobs die: a failed attempt is `job_executions.status = FAILED`, while the job moves to `RETRYING` (transient) or `DEAD_LETTER` (permanent). `FAILED` survives as a job status for queues that opt out of the dead letter queue.

On the brief — The brief also writes the lifecycle as `Queued → Scheduled → Claimed`. Causally, `SCHEDULED` precedes `QUEUED` — a future job becomes eligible, then gets picked up. Both readings are noted here as decisions rather than oversights.

5.2 The conditional write

Every transition out of an in-flight state carries a guard. This single clause is what makes the lease scheme safe:

```
UPDATE jobs SET status = ...
WHERE id = $1
  AND worker_id = $me      -- I still own it
  AND status = 'RUNNING'   -- it is still where I left it
```

Zero rows updated means **"I no longer own this job"** — the reaper reclaimed it while this worker was slow. The worker abandons its result rather than writing it, increments a duplicate-detection counter, and logs the event.

5.3 Crash recovery

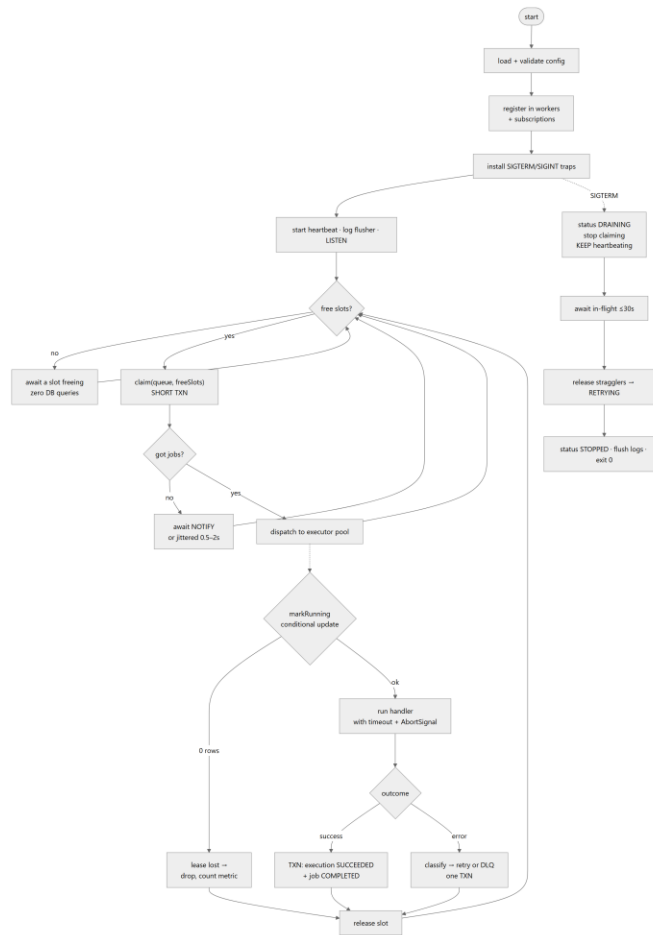


Figure 4 — Worker process flow

Crash point	Job state	Recovered by	Latency
Before claim commits	QUEUED	Nothing needed — the transaction rolled back	0
After claim, before RUNNING	CLAIMED	Reaper, on lease expiry	≤ 60 s
Mid-execution	RUNNING	Reaper, on lease expiry	≤ 60 s
After the side effect, before the commit	RUNNING	Reaper — job retried, side effect repeats	≤ 60 s
After completion commits	COMPLETED	Nothing needed	—

Row four is irreducible. It is why the honest guarantee is **at-least-once**, not exactly-once — see §8.

5.4 Heartbeats and the timing invariant

Timer	Value	Meaning
Heartbeat interval	5 s	How often a worker reports itself alive
Worker timeout	30 s	Six missed beats — the worker is marked DEAD
Lease / visibility timeout	60 s	After this, a claimed job is reclaimable

Load-bearing — The lease must exceed the worker timeout. If it expired first, the reaper would reclaim jobs from a worker that is merely six seconds slow — causing exactly the duplicate execution the design prevents. The 2× gap absorbs GC

pauses, brief network hiccups and clock skew, and the invariant is validated at boot: a configuration that violates it refuses to start.

Each heartbeat also renews the lease on every job the worker holds. Without renewal, a five-minute job under a sixty-second lease would be reclaimed four times and run five times concurrently. With it, the lease means "this worker is alive and still holds this job" rather than "this job must finish within sixty seconds".

5.5 Graceful shutdown

On SIGTERM the worker:

- Sets its state to DRAINING and stops claiming new work.
- **Keeps heartbeating and keeps renewing leases.** This is the step most implementations miss — stop here and the reaper reclaims your in-flight jobs while you are still running them, causing the duplicate execution you designed against, on every deploy.
- Awaits in-flight jobs, bounded by a 30-second grace period.
- For anything still running at the deadline: aborts it, then releases the lease explicitly so another worker picks it up in ~0 s rather than waiting out the full lease.
- Marks itself STOPPED, flushes buffered logs, and exits 0.

Important — *SIGKILL runs none of this. That is precisely why the reaper exists: graceful shutdown is an optimisation, the lease is the guarantee. The system is therefore tested with docker kill, not docker stop.*

6. Retries and backoff

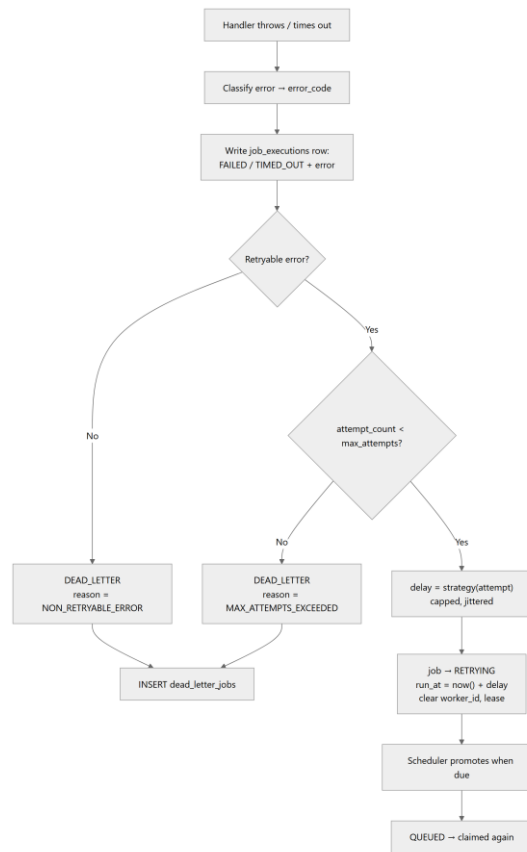


Figure 5 — Retry decision flow

6.1 The three strategies

With base = 5 s, max = 300 s, max_attempts = 5:

Attempt fails	FIXED	LINEAR	EXPONENTIAL	with $\pm 10\%$ jitter
1 → wait	5 s	5 s	5 s	4.5 – 5.5 s
2 → wait	5 s	10 s	10 s	9 – 11 s
3 → wait	5 s	15 s	20 s	18 – 22 s
4 → wait	5 s	20 s	40 s	36 – 44 s
5	DEAD LETTER	DEAD LETTER	DEAD LETTER	DEAD LETTER

6.2 Jitter is not decoration

A downstream API fails for sixty seconds and 500 jobs fail within the same second. Without jitter, all 500 retry at exactly $t+5$ s, then $t+15$ s, then $t+35$ s — a synchronised thundering herd that re-saturates the service the instant it recovers, likely knocking it over again.

With $\pm 10\%$ jitter the retries smear across a window, and the recovering service sees a ramp instead of a wall. Two lines of code separate a self-healing system from a self-perpetuating outage.

6.3 The worker does not sleep

On failure the worker computes the next `run_at`, writes it, and frees its concurrency slot immediately. **Sleeping through the backoff would hold a slot for the entire window** — the most common design mistake in this problem.

6.4 Not every failure deserves five attempts

Error class	Retryable	Reasoning
TIMEOUT, HTTP 5xx, <code>ECONNREFUSED</code> , <code>ETIMEDOUT</code>	Yes	Transient — downstream or transport
RATE_LIMITED (429)	Yes	Retry, honouring Retry-After as a backoff floor
HTTP 4xx (except 408, 429)	No	A malformed request stays malformed
VALIDATION_ERROR	No	Bad payload
UNKNOWN_HANDLER	No	Misconfiguration, not a transient fault
Unrecognised errors	Yes	Fails safe. An unknown error is more likely transient than permanent, and <code>max_attempts</code> bounds the cost of being wrong.

Retrying a 400 Bad Request four more times burns 75 seconds and four concurrency slots to reach a conclusion available on attempt one.

7. Dead letter queue

The dead letter queue is not an error log. It is a work item requiring a human decision: fix the input and replay, or accept the loss. Designing it as an inbox with a resolution workflow, rather than a table of failures, is the difference between a feature and a checkbox.

7.1 What happens, transactionally

One transaction, three writes:

- The execution row is closed as `FAILED` or `TIMED_OUT` with its error and duration.
- The job moves to `DEAD_LETTER`, its lease and worker cleared.
- A `dead_letter_jobs` row is created with the reason, error, attempt count and a payload snapshot.

Design — *The original job row is kept, not moved. That keeps `job_id` foreign keys valid, keeps the execution history reachable, and means the job detail page works identically for a dead-lettered job. The DLQ table is an index over failures plus resolution metadata — not a second home for the job.*

7.2 Triage by error signature

Four hundred dead-lettered jobs are usually three problems. The inbox groups by a normalised error signature — ids, timestamps, URLs and numbers stripped — so identical failures collapse into one group with a count, first and last seen, and affected queues. That turns a table dump into a triage list.

7.3 Replay creates a new job

A replay **never resurrects the original**. It creates a new job with `parent_job_id` set, from the payload snapshot, with a fresh attempt count.

- **The audit trail survives.** "This failed five times, was replayed with a corrected payload, and succeeded" stays visible. Resetting `attempt_count` on the original would erase the history the executions table exists to capture.
- **The chain is queryable.** A job replayed three times, each with a different payload, is fully traceable.
- **It is idempotent.** A `WHERE resolved_at IS NULL` guard means a double-clicked Replay button produces exactly one job; the second call returns 409 naming the existing replay.

8. Idempotency

Two different duplicates are commonly conflated. They have completely different solutions.

8.1 Duplicate submission — fully solvable

A client POSTs a job, the API commits it, and the response is lost to a network timeout. The client retries. Without protection, two jobs now exist.

Solved at the database level with a partial unique index on (queue_id, idempotency_key). A retried request carrying the same key returns the original job with 200 and X-Idempotent-Replay: true, rather than creating a second.

Deliberate — *Idempotency is opt-in via an explicit key, never inferred from a payload hash. Two identical jobs submitted deliberately — "send the daily report", twice — must both run. Content-hash deduplication would silently swallow the second and be maddening to debug. Choosing not to auto-deduplicate is the more considered decision.*

8.2 Duplicate execution — not fully solvable

Exactly-once execution is impossible in a system whose side effects are external to its database.

The worker cannot atomically "send the email" and "record that the email was sent" — those are two systems with no shared transaction. Whichever order is chosen, a crash in the gap produces either a duplicate or a lost job. Choosing at-least-once means choosing duplicates over losses, which is the right trade for a job runner.

Stating this plainly is worth more than claiming exactly-once. Three layers mitigate it:

Layer	Responsibility	Mechanism
Database	Make duplicate submission and duplicate attempt records impossible	Partial unique index on idempotency keys; UNIQUE (job_id, attempt) on executions
Worker	Shrink the window; never knowingly double-run	Conditional writes on every transition; lease renewal; a duplicate_execution_detected counter
Handler	Be safe if invoked twice	Receives a stable execution token (the job id). The built-in HTTP handler sends it downstream as an Idempotency-Key header

Not built — *The transactional outbox pattern, two-phase commit, and a dedupe-token table with TTLs are all real solutions to this problem, and all three are more machinery than this scope can carry. They are named here as considered-and-rejected rather than overlooked.*

9. Design decisions and trade-offs

Each decision below states the alternative that was rejected and what the choice costs. A trade-off with no stated cost is not a trade-off.

9.1 PostgreSQL as the queue, not Redis or RabbitMQ

A broker is faster — tens of thousands of jobs per second — but splits the system in two. Creating a job means writing to PostgreSQL and pushing to the broker, with no transaction spanning both. Crash in between and a job exists in the UI but never runs, or runs but was never recorded. The standard fixes (transactional outbox, CDC) are more machinery than this entire assignment. PostgreSQL gives atomic creation, atomic claiming, transactional cron materialisation, and one backup. SKIP LOCKED exists for exactly this pattern.

Cost: *A ceiling around 1,000–5,000 jobs/second, and more write amplification than a purpose-built broker. Four orders of magnitude above what this system needs.*

9.2 Polling and push, not either alone

Polling alone wastes queries and adds latency. Push alone is unsafe: a notification delivered while a worker reconnects is gone forever. Using both makes push an optimisation over a correct baseline.

Cost: *Two code paths, and a dedicated non-pooled connection per worker for LISTEN.*

9.3 Separate worker processes, not in-process threads

Different scaling axis, different failure domain, different lifecycle. It is also the only way to demonstrate distributed behaviour, which the brief explicitly requires.

Cost: *More moving parts in Compose, and shared code must live in a package rather than being casually imported.*

9.4 Leader election, not a dedicated scheduler deployable

A separate service works until somebody scales it to two and every cron fires twice. Leader election makes correctness structural rather than procedural, and gives automatic failover when the leader's session dies.

Cost: *Roughly forty lines of election logic, and one failover test.*

9.5 Advisory lock, not lock-free concurrency counting

The lock-free version is wrong: two workers reading the same MVCC snapshot both see zero running and both claim full capacity. SKIP LOCKED cannot help — the conflict is over an aggregate, not over rows.

Cost: *Claims on a single queue serialise, capping that queue at roughly 300–1000 claim transactions per second. Each claim takes a batch, so effective throughput is far higher.*

9.6 Short JWT in memory plus a rotating refresh cookie

A long-lived token in localStorage is the common shortcut and is XSS-exposed — any injected script reads it. Server sessions require a database hit per request. This keeps the access token out of any persistent store and the refresh token unreachable from JavaScript.

Cost: *Refresh-rotation logic in the API client, including collapsing concurrent 401s into a single refresh.*

9.7 Retry policy snapshotted onto each job

Reading the queue's policy live means editing it silently rewrites the contract of thousands of in-flight jobs, including ones already mid-backoff — and makes historical behaviour unexplainable.

Cost: Five denormalised columns on jobs, and a policy edit does not apply retroactively. That is the intended behaviour, surfaced in the UI.

9.8 Cursor pagination, not offset

On a table taking thousands of inserts a minute, offset pagination is incorrect, not merely slow: rows shift between requests, so callers see duplicates and miss records. Deep offsets also force PostgreSQL to scan and discard everything before the window.

Cost: No "jump to page 7". Acceptable — nobody jumps to page 7 of a job list; they filter.

9.9 Attempts counted at claim, not at completion

Counting at completion means a job that crashes its worker before recording anything is reclaimed forever — a poison pill that kills the fleet one process at a time. Counting deliveries, as SQS does, bounds the blast radius.

Cost: A job whose worker died before it ever ran still burns an attempt. Surfaced honestly in the UI as "Attempt 1 — never started (worker lost)".

9.10 A promotion loop, not a smarter claim query

The alternative — matching SCHEDULED and RETRYING directly in the claim query — is simpler and was seriously considered. Promotion won because the partial claim index then covers only truly-ready rows rather than every future-dated and backing-off job, which is the entire performance argument for that index.

Cost: Up to one second of promotion latency, and the scheduler becomes necessary for timely execution — though never for correctness. Jobs fire late, never never.

9.11 Deliberately not built

Feature	Why not, and what it would take
Queue sharding	Invisible below roughly 10,000 jobs/second. Would require a shard key so workers claim disjoint slices without a shared lock. Days of work to demonstrate nothing at demo scale.
Workflow DAGs	A genuine multi-day feature — cycle detection, partial-failure semantics, cascading cancellation. A half-working DAG engine reads worse than none. parent_job_id already exists as the extension point.
Multi-region / replicas	Not assessable locally, and not demonstrable in the scope of this submission.
A real handler ecosystem	The brief permits simulated handlers. Every third-party integration is credentials and flakiness for no marks.

10. API design

56 operations across 9 resource groups, versioned at `/api/v1`. Interactive documentation is served at `/docs` and the OpenAPI 3.0 specification is exported to `docs/api/openapi.json`.

Group	Ops	Covers
auth	5	Register, login, refresh, logout, current principal
projects	12	Organizations, projects, retry policies, API keys
queues	8	CRUD, pause/resume, live statistics
jobs	9	Create (4 timing modes), batch, explorer, detail, executions, logs, cancel, retry
schedules	9	Cron CRUD, validation preview, pause/resume, manual trigger, run history
workers	3	Fleet health, detail, heartbeat series
dlq	5	Inbox, grouping by error signature, replay, discard
metrics	3	Overview, throughput series, latency percentiles
health	2	Liveness and readiness (outside the version prefix)

10.1 Conventions

- **Two auth schemes.** Bearer JWT for the dashboard; hashed API keys for services. Both resolve to the same principal.
- **Deny by default.** Every route requires a principal unless explicitly marked public. Opt-in auth is how endpoints ship unprotected.
- **Cross-tenant access returns 404, not 403.** Returning 403 would confirm the id exists, turning the API into an oracle for probing another tenant's data.
- **One error envelope, always.** A stable machine-readable code, a human-readable message that may change, per-field details, and a `request_id` that correlates to the server log and to every job the request created.
- **Validation at submission.** A payload that cannot succeed is rejected with 422 while the caller still holds the request — rather than being accepted, queued, claimed, executed, failed, retried four times and dead-lettered twenty minutes later.

10.2 No POST `/workers`

Workers register themselves through the database, not over HTTP. A worker that can reach only PostgreSQL remains fully functional, which keeps the API entirely off the critical path of job execution. That is a real availability property, not an implementation detail.

11. Testing strategy

Five marks on paper, but the evidence for the thirty-five marks of reliability and backend engineering. An untested concurrency claim is an assertion; a test is a proof.

11.1 Unit tests — 85, no database, under one second

- **All 81 (from, to) status pairs** asserted against the declared transition table, pinned by an inline snapshot so an unconsidered edge shows up as a diff.
- **Jitter bounds over 1,000 samples**, plus a spread assertion — proving jitter actually desynchronises retries rather than merely existing.
- **Both DST edge cases**: a 02:30 daily job fires exactly once on the spring-forward day (when 02:30 does not exist) and exactly once on the fall-back day (when it occurs twice).
- All three misfire policies verified against a simulated 30-minute scheduler outage.

11.2 Concurrency tests — the gate

Mandatory, not preferred — *These run against a real PostgreSQL via Testcontainers. An in-memory or mocked database cannot exhibit FOR UPDATE SKIP LOCKED semantics, row locks, or MVCC snapshots — so a mocked concurrency test proves precisely nothing about the property it claims to verify.*

Test	Asserts
Exactly-once claiming	20 concurrent claimers over 500 jobs: 500 claims, 0 duplicates, every attempt_count exactly 1
End-to-end exactly-once	10 workers × 5 slots over 300 jobs: exactly 300 execution rows, none duplicated, all COMPLETED
Per-queue concurrency	The limit is never exceeded — and is demonstrably saturated. The second assertion matters as much as the first: a claim that admitted nothing would satisfy the ceiling trivially
Priority ordering	A future-dated CRITICAL job never preempts a ready BULK one — the trap in the brief
Crash recovery	SIGKILL mid-job; the reaper recovers, another worker finishes, exactly two execution rows exist
The zombie worker	A revived worker's completion write affects zero rows and its result is discarded
Cron exactly-once	Two schedulers materialising the same due schedule concurrently create exactly one job
Leader failover	A new leader within five seconds of the previous one releasing

The concurrency suite is run repeatedly in CI. Race conditions are probabilistic — a single green run is not evidence.

11.3 Defects found by this discipline

Nine defects were caught before any of this ran in anger. Four illustrate the value:

- **Error signatures did not normalise "300ms"**. The pattern used a word boundary, and there is none between digits and a trailing unit — so two identical failures produced different signatures and would have landed in separate DLQ groups. Exactly the failure the grouping exists to prevent.
- **The Prisma client was missing from the runtime image**. Generated during the build stage, but the runtime stage copied node_modules from an earlier stage. The image built fine and died on boot — in the container only.

- **Host clock versus database clock.** Stamping `run_at` from the application and comparing it against the database's `now()` made a just-created job briefly not-yet-due. It surfaced as a one-in-ten test flake, which is precisely what repeated runs are for.
- **POST endpoints falsely reported 201 Created.** Pause, resume and discard create nothing. A client written against the generated specification would have branched on the wrong code.

12. Running the system

12.1 One command

```
docker compose up -d --build
```

Starts seven containers: PostgreSQL, the API, the scheduler, three independent workers, and the dashboard. Then register an account at <http://localhost:5173> and run:

```
npm run seed
```

This creates a demo organization with four queues of differing character, two cron schedules, and roughly 58 jobs, which the workers begin draining immediately.

Surface	URL
Dashboard	http://localhost:5173
API documentation (Swagger)	http://localhost:3000/docs
Prometheus metrics	http://localhost:3000/metrics
Health / readiness	http://localhost:3000/health · /ready

12.2 The demonstration

Open the Workers page, then:

```
docker kill djs-worker-2
```

SIGKILL — instant termination, no drain, no lease release, equivalent to a server losing power mid-job. Then watch:

- **At roughly 30 seconds** the scheduler marks the worker DEAD after six missed heartbeats.
- **At roughly 60 seconds** the reaper finds its expired leases, closes those attempts as ABANDONED, and requeues the jobs.
- **Immediately after** the surviving workers claim and complete them.

Nothing is lost, and nobody intervenes. Contrast with `docker stop`, which sends SIGTERM: the worker finishes its in-flight jobs and exits cleanly in a few seconds, with no job ever retried.

12.3 Verified state

Check	Result
Backend type checking	0 errors
Frontend type checking	0 errors
Unit tests	85 / 85 passing
Migrations against PostgreSQL 16	Applied clean on first attempt
Claim query plan	Index Scan, no Sort node, 0.098 ms over 50,000 rows
Containerised SIGTERM drain	All in-flight jobs COMPLETED, worker exited 0 in 3.5 s
Containerised SIGKILL recovery	All jobs recovered and completed by surviving workers

12.4 Where the detail lives

Document	Contents
README.md	Quick start, commands, troubleshooting
docs/SETUP.md	Step-by-step setup written for a first-time Docker user
docs/ARCHITECTURE.md	The full design — 30 sections, including all 30 failure scenarios and transaction boundaries
docs/API.md	Complete API reference with request and response bodies
docs/DATABASE.md	ER diagram and table-by-table schema reference
docs/DESIGN-DECISIONS.md	The trade-off register
docs/VERIFICATION.md	Measured evidence for every claim made in this document

End of document